

Automata & Regular Expressions & Languages① Introduction to finite Automata :-

- Automata theory is the study of abstract computing devices, or "machines".
- Finite Automata & certain kinds of formal grammars, are used in the design & construction of ~~imp~~ kinds of S/w.
- FA are a useful model for many important kinds of H/w & S/w. Some of the most imp kinds are.

* / S/w for designing & checking the behavior of digital circuits.

* The "lexical Analyzer" of a typical compiler, that is, the compiler component that breaks the input text into logical units, such as identifiers, keywords & punctuation.

* Slw for scanning large bodies of text, Such as collections of web pages, to find occurrences of words, phrases, or other patterns.

* Slw for verifying Systems of all types that have a finite no of distinct states, Such as communications protocols or protocols for secure exchange of info.

→ The advantage of having only a finite no of states is that we can implement the System with a fixed set of resources.

→ The purpose of a state is to remember the relevant portion of the System's history.

Eg (1) Simple finite Automata is an on/off switch

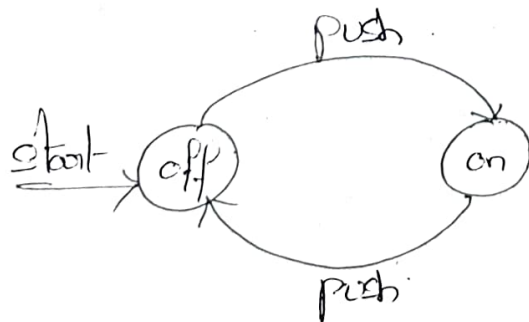


Figure 0-1
A FA modeling an on/off switch.

→

→ A.

the

→ one

state

is f

→ Can

work

state

→ at is

state

→ for

state

that of

switch

key

→ In FA, the States are represented by circles.
(Here states are on & off).

→ Arrows bn states are labeled by "inputs",
which represent external influences on the
System. (Here, both arcs are labeled by
the flip push).

→ one of the states is designated the "Start
State", the state in which the system
is placed initially. (Here Start State is off).

→ Conventionally indicate the start state by the
word start & an arrow leading to that
state.

→ It is often necessary to indicate one or more
states as "final" or "accepting" states

→ for instance, we could have regarded the
state on as accepting state, because in
that state, the device being controlled by the
switch will operate.

x.g

3ms

e

1.

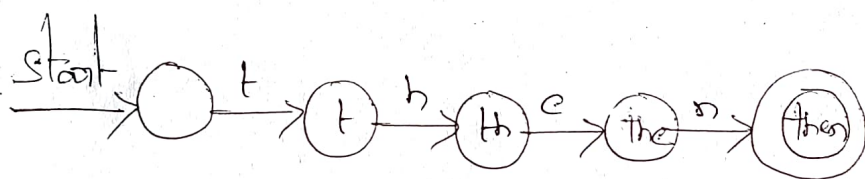
with

g

switch

→ The "final & accepting" state is indicated by double circle.

Eg :- (2) FA that could be part of a lexical Analyzer. The job of this automation is to recognize the keyword "then".



→ The start state corresponds to the empty string & each state has a transition on the next letter of then to the state that corresponds to the next - longer - prefix.

(2) The central concepts of Automata Theory :-

→ The central concepts include the "alphabet" (a set of symbols), "strings" (a list of symbols from an alphabet), & "language" (a set of strings from the same alphabet).

(a)

An "

Symbol

Eg :-

1. Σ

2. Σ

3. The

all

(b) String

A String

from some

Eg :-

① call is

② " "

(9) The

The con

(a) Alphabets :-

An "Alphabet" is a finite, nonempty set of symbols. It is denoted by Σ .

Eg :-

1. $\Sigma = \{0, 1\}$, the binary Alphabet.
2. $\Sigma = \{a, b, \dots, z\}$, the set of all lowercase letters.
3. The set of all ASCII characters, or the set of all printable ASCII characters.

(b) Strings :-

A String is a finite sequence of symbols chosen from some alphabet.

Eg :-

① 01101 is a string from binary alphabet $\Sigma = \{0, 1\}$.

② 111

(i) The empty String :-

The empty string is the string with zero -

occurrences of symbols. This string is denoted by 'ε'.

(ii) Length of a string :-

→ Length of a string is the no of positions for symbols in the string.

Eg :-

String is 01101, length is 5.

→ The standard notation for the length of a string is ω is |ω|.

Eg :-

|011| = 3 and |ε| = 0.

(iii) Power of an Alphabet :-

if 'Σ' is an alphabet, we can express the set of all strings of a certain length from that alphabet by using an exponential notation. we define

Σ^k to be the set of strings of length k,

Each σ denoted

Eg :-

Σ⁰ = { }

if Σ

Σ¹ = { }

Σ³ = { }

→ The set denoted

Eg :-

{0,1}*

Σ*

→ Set of strings from the alphabet

Σ⁺

denoted

each of whose symbols is in Σ' .

Eg :-

$\Sigma^0 = \{\epsilon\}$, that is ϵ is the only string whose length is '0'.

as for

If $\Sigma = \{0, 1\}$, then

$\Sigma^1 = \{0, 1\}$, $\Sigma^2 = \{00, 01, 10, 11\}$

$\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$.

after

→ The set of all strings over an alphabet Σ is denoted by Σ^* .

Eg :-

$\{0, 1\}^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}$.

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

set of
alphabet
free.

→ Sometimes, we wish to exclude the empty string from the set of strings. The set of nonempty strings from alphabet Σ is denoted by Σ^+ .

Σ^+

Thus, two appropriate equivalences are:

$$* \Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

$$* \Sigma^* = \Sigma^+ \cup \{\epsilon\}.$$

(iv) Concatenation of strings $\circ$

Let x & y be the two strings. Then " xy " denotes the concatenation of x & y .

Eg $\circ$

$$\textcircled{1} x = a_1 a_2 \dots a_n$$

$$y = b_1 b_2 \dots b_m \quad \text{Then}$$

$$xy = a_1 a_2 \dots a_n b_1 b_2 \dots b_m.$$

$$\textcircled{2} x = 01101, y = 110$$

$$xy = 01101110.$$

Note $\circ$

for any string w , the ϵ

$$\boxed{\epsilon w = w\epsilon = w} \text{ hold}$$

(c) Languages

→ A set of strings all of which are chosen from some Σ^* , where Σ is a particular alphabet, is called a Language.

→ If Σ is an alphabet, & $L \subseteq \Sigma^*$, then L is a language over Σ .

→ Language can be viewed as a set of strings

Eg.

① English is a Language, whose collection of legal English words is a set of strings over the alphabet that consists of all the letters.

② The Language of all strings consisting of n 0's followed by n 1's, for some $n \geq 0$.

Ans. $\{ \epsilon, 01, 0011, 000111, \dots \}$.

③ The set of strings of 0's & 1's with an equal no. of each.

Ans. $\{ \epsilon, 01, 10, 0011, 0101, 1001, \dots \}$

④ The Set of binary no whose value is a prime.

Ans. $\{10, 11, 101, 111, 1011, \dots\}$.

⑤ Σ^* is a language for any alphabet Σ .

⑥ $\emptyset$, the empty language, is a language over any alphabet.

⑦ $\{\epsilon\}$, the language consisting of only the empty string, is also a lang over any alphabet.

Note :-

$$\emptyset \neq \{\epsilon\}.$$

↓ ↓
No String one String.

① problems :-

→ In automata theory, a problem is the question of deciding whether a given string is a member of some particular language.

→ If Σ' is an alphabet, & L' is a language over Σ' , then the problem L' is:

* Given a string w in Σ^* , decide whether or not w is in L .

eg :-

The prob of testing primality can be expressed by the lang L_p consisting of all binary strings whose value as a binary no is a prime.

That is, given a string of 0's & 1's, say "yes" if the string is the binary representation of a prime & say "no" if not.

Finite Automata :-

→ A finite Automata has a set of states, & its "control" moves from state to state in response to external "input".

→ one of the crucial distinctions among classes of finite Automata is whether that control is "deterministic", meaning that the automata

cannot be in more than one state at any one time. or "nondeterministic", meaning that it may be in several states at once.

④ Deterministic Finite Automata

→ The term "deterministic" refers to the fact that on each input there is one & only one state to which the automaton can transition from its current state.

⑤ Definition of a DFA.

A DFA consists of:

1. A finite set of states, often denoted by Q .
2. A " " " input symbols, often denoted by Σ .
3. A Transition function that takes as arguments a state & an input symbol & returns a state. It is denoted by δ .

4. A Start State, one of the states in Q .
5. A Set of final or accepting states F . The set F is a subset of Q .

→ A deterministic finite Automata often be referred as "DFA".

→ The representation of DFA consists of 5 ~~parts~~ components, & the notation is

$$A = (Q, \Sigma, S, \gamma, F)$$

where A is the name of the DFA.

(a) How a DFA processes strings

(b) Simple notation for DFA

→ Specifying a DFA as a 5 tuple with a detailed description of the γ transition function is both tedious & hard to read. There are two preferred notations for describing automata.

1. A Transition diagram, which is a graph.
2. A Transition Table, which is a tabular listing of the δ function, which by implication tells us the set of states of the F/P alphabet.

1. Transition Diagram $\circ$ (TD)

A transition diagram for a DFA $A = (Q, \Sigma, \delta, q_0, F)$ is a graph defined as follows:

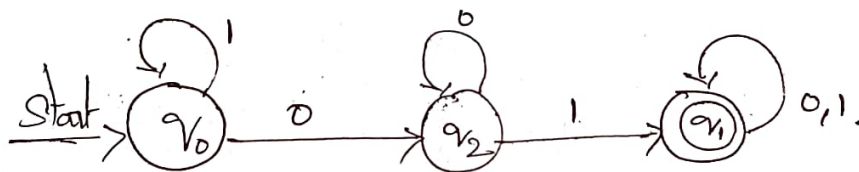
- a). for each state in Q there is a node.
- b). for each state q in Q & each F/P symbol a in Σ , let $\delta(q, a) = p$. Then the transition diagram has an arc from node q to node p , labeled a .

If there are several F/P symbols that cause transitions from q to p , then the TD can have one arc, labeled by the list of these symbols.

c). There is an arrow into the Start state q_0 labeled by Start. This arrow does not originate at any node.

d). Nodes corresponding to accepting states (those in F) are marked by double circle. States not in F have a single circle.

Eg :- The DFA accepting all strings with a substring 01.
 Σ, δ, q_0, F



2. Transition Table :-

→ A transition Table is a conventional tabular representation of a function like δ that takes two arguments & returns a value.

→ The rows of the table correspond to the states & the columns correspond to the inputs.

→ The entry for the row corresponding to state q & the column corresponding to input a is the state $\delta(q, a)$.

Eg :-

	0	1
→ q_0	q_2	q_0
q_1	q_1	q_1
q_2	q_2	q_1

(c) How a DFA processes strings :-

A string x is accepted by a finite automata

$$M = (Q, \Sigma, \delta, q_0, F)$$

if $\delta(q_0, x) = q$ for some $q \in F$.

This is basically the acceptability of a string by the final state.

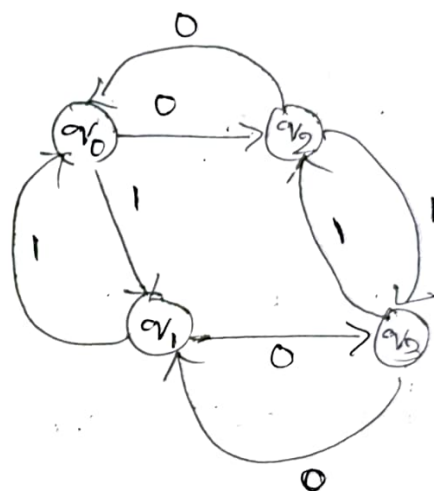
Eg :-

Consider the FA whose transition function is δ given by below table in the form of

Transition table. Here $Q = \{q_0, q_1, q_2, q_3\}$,
 $\Sigma = \{0, 1\}$, $F = \{q_0\}$. Give the entire
 Sequence of states for the IP string 110101.

Table :-

	0	1
q_0	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2



Ans :-

$$\delta(q_0, 110101) = \delta(q_1, 10101)$$

$$= \delta(q_0, 0101)$$

$$= \delta(q_2, 101)$$

$$= \delta(q_3, 01)$$

$$= \delta(q_1, 1)$$

$$= \delta(q_0, \epsilon)$$

$$= q_0$$

Hence

$$q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_0 \xrightarrow{0} q_2 \xrightarrow{1} q_3 \xrightarrow{0} q_1 \xrightarrow{1} q_0$$

$\Rightarrow$ The Symbol '↓' indicates that the current
 if ~~starting~~ Symbol is being processed by
 the machine. So the above string is accepted by
 FA, because q_0 is final state.

eg $\begin{smallmatrix} 0 \\ 0 \end{smallmatrix}$ (i) 110001 (ii) 1111001

(d) Extending the Transition function to Strings

$\rightarrow$ we define an extended transition function
 that describes what happens when we start
 in any state & follow any sequence of
 i/p's.

$\rightarrow$ If δ is our transition function, then the
 extended transition function constructed from
 δ will be called $\hat{\delta}$.

$\rightarrow$ The extended Transition function is a function
 that takes a state q & a string w &
 returns a state p .

Basis:

Induction:

eg $\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}$

all but

eg $\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}$

Then $\hat{\delta}$

eg $\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}$

reach

Transitions

→ we define $\hat{\delta}$ by induction, on the length of the input string, as follows:

Basis: $\hat{\delta}(q, \epsilon) = q$: — That is, if we are in state 'q' & read no input, then we are still in state q.

Induction: Suppose 'w' is a string of the form xa ; that is, a is the last symbol of 'w', & 'x' is the string consisting of all but the last symbol.

eg: — ① $w = 1101$ is broken into $x = 110$ & $a = 1$.

Then
$$\hat{\delta}(q, w) = \delta(\hat{\delta}(q, x), a)$$

eg: — ② String $w = 110101$. Verify this string reaches the final state or not. for the Transition Table is below.

	0	1
q_0	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

Ans:-

The check involves computing $\hat{\delta}(q_0, w)$ for each prefix w of 110101, starting at ϵ & going in increasing size.

- $\hat{\delta}(q_0, \epsilon) = q_0$
- $\hat{\delta}(q_0, 1) = \delta(\hat{\delta}(q_0, \epsilon), 1) = \delta(q_0, 1) = q_1$
- $\hat{\delta}(q_0, 11) = \delta(\hat{\delta}(q_0, 1), 1) = \delta(q_1, 1) = q_0$
- $\hat{\delta}(q_0, 110) = \delta(\hat{\delta}(q_0, 11), 0) = \delta(q_0, 0) = q_2$
- $\hat{\delta}(q_0, 1101) = \delta(\hat{\delta}(q_0, 110), 1) = \delta(q_2, 1) = q_3$

$\hat{\delta}(q_0,$

finally we

(e) The L

$\rightarrow$ we can

$A =$

denoted by

$L(A) =$

$\rightarrow$ That is,

strings

one of

$\rightarrow$ If L

Say L

$$\hat{\delta}(q_0, 11010) = \delta(\hat{\delta}(q_0, 1101), 0) = \delta(q_3, 0) = q_1$$

$$\hat{\delta}(q_0, 110101) = \delta(\hat{\delta}(q_0, 11010), 1) = \delta(q_1, 1) = q_0$$

finally we reached the final state q_0 .

(c) The Language of a DFA.

→ we can define language of a DFA

$A = (Q, \Sigma, \delta, q_0, F)$. ~~This~~ language is

denoted by $L(A)$, & is defined by

$$L(A) = \{ w / \hat{\delta}(q_0, w) \text{ is in } F \}$$

→ That is, the language of 'A' is the set of strings w that take the start state q_0 to one of the accepting states.

→ If 'L' is $L(A)$ for some DFA A, then we say 'L' is a regular language.

Non Deterministic Finite Automata

→ NFA has the power to be in several states at once.

(a) An Informal View of NFA

→ The diff b/w the DFA & NFA is in the type of δ .

→ For the NFA, δ is a function that takes a state & input symbol as arguments, but returns a set of zero, one, or more states.

eg

A NFA, whose job is to accept all & only the strings of 0's & 1's that end in 01.

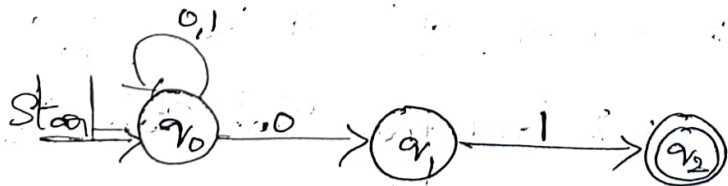


Figure (a) - An NFA accepting all strings that end in 01.

→ state
happy
accept
only
q0 →

Figure -
the power

→ So for the

(b) Def

An NFA

→ The following figure shows how an NFA processes I/p's. we have shown what happens when the automaton of figure ① receives the ~~for~~ I/p Sequence 00101. It only stays in start state q_0 .

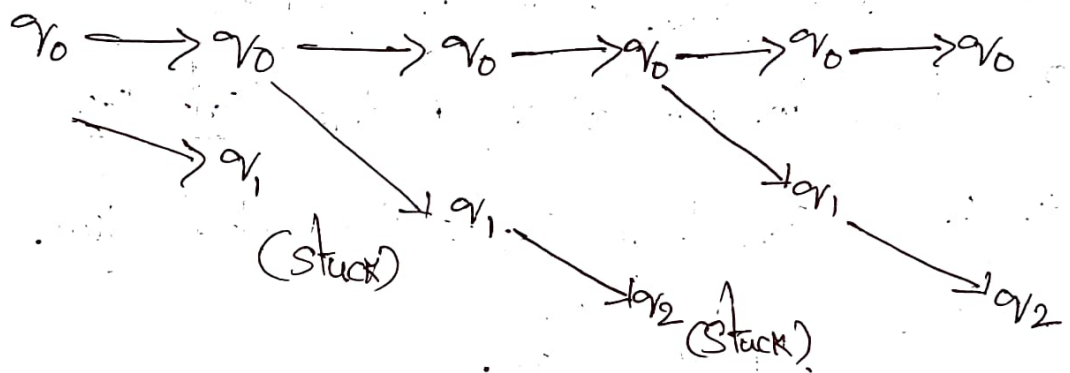


Figure: — (b). The states an NFA is in during the processing of I/p Sequence 00101.

→ So this string is accepted, because it reached the final state q_2 .

⑥ Definition of NFA :-

An NFA is represented as follows:

$$A = (Q, \Sigma, \delta, q_0, F)$$

where:

1. Q is a finite set of States.
2. Σ is a finite " " inp Symbols.
3. q_0 , a member of Q , is the Start State.
4. F , a subset of Q , is the set of final (or accepting) States.
5. δ , the transition function, is a function that takes a state in Q , & an inp Symbol in Σ as arguments & returns a subset of Q .

Eg :- The NFA of figure (a) can be specified formally as

$$(\{ q_0, q_1, q_2 \}, \{ 0, 1 \}, \delta, q_0, \{ q_2 \})$$

where the transition function δ is given by the

→ q_0
 q_1
 $* q_2$
 → The a
 table
 is a
 → Also
 at all
 Symbol
 empty
 © The
 → we m
 a func
 of inp
 states -

Transition Table of Figure (a):

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$* q_2$	$\emptyset$	$\emptyset$

$\rightarrow$ The only diff is that each entry in the table for the NFA is a set, even if the set is a 'Singleton' (has one member).

$\rightarrow$ Also notice that when there is no transition at all from a given state on a given IP Symbol, the proper entry is $\emptyset$, the empty set.

© The Extended Transition function:

$\rightarrow$ we need to extend the tf δ of an NFA to a function $\hat{\delta}$ that takes a state q & a string of IP Symbols w , & returns the set of states that the NFA is in if it starts in

state q & processes the string w .

→ formally, we define $\hat{\delta}$ for an NFA's transition function δ by:

Basis:-

$\hat{\delta}(q, \epsilon) = \{q\}$, that is without reading any IP symbols, we are only in the state we began in.

Induction:-

Suppose w is of the form $w = xa$, where 'a' is the final symbol of w & x is the rest of w .

Also suppose that $\hat{\delta}(q, x) = \{p_1, p_2, \dots, p_k\}$.

Let

$$\bigcup_{i=1}^k \delta(p_i, a) = \{a_1, a_2, \dots, a_m\}$$

Then $\hat{\delta}(q, w) = \{a_1, a_2, \dots, a_m\}$. we first compute $\hat{\delta}(q, w)$ by first computing $\hat{\delta}(q, x)$ & by then following any transition from any of these states that is labeled a .

Eg:-

Processing

1. $\hat{\delta}(q_0, \epsilon)$

2. $\hat{\delta}(q_0, a)$

3. $\hat{\delta}(q_0, aa)$

4. $\hat{\delta}(q_0, aaa)$

5. $\hat{\delta}(q_0, aaaa)$

6. $\hat{\delta}(q_0, aaaaa)$

Q.1) The L

formally,
NFA, the

Eg :- Let us use $\hat{\delta}$ to determine describe the processing of IP 00101 by the NFA.

$$1. \hat{\delta}(q_0, \epsilon) = \{q_0\}.$$

$$2. \hat{\delta}(q_0, 0) = \delta(q_0, 0) = \{q_0, q_1\}$$

$$3. \hat{\delta}(q_0, 00) = \delta(q_0, 0) \cup \delta(q_1, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$$

$$4. \hat{\delta}(q_0, 001) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$$

$$5. \hat{\delta}(q_0, 0010) = \delta(q_0, 0) \cup \delta(q_2, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$$

$$6. \hat{\delta}(q_0, 00101) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$$

(d) The Language of an NFA :-

formally, if $A = \{Q, \Sigma, \delta, q_0, F\}$ is an NFA, then

$$L(A) = \{w / \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$$

That is, $L(A)$ is the set of strings w in Σ^* such that $\delta(q_0, w)$ contains at least one accepting state.

② Equivalence of Deterministic & Non-deterministic Finite Automata :

→ Every Language that can be described by some NFA can also be described by some DFA.

→ In the worst case the DFA can have 2^n States while the NFA for the same language has only n States.

→ The proof that DFA's can do whatever NFA's can do involves an imp "construction" called the Subset construction because it involves constructing all Subsets of the set of states of the NFA.

Subset construction :

→ Subset construction starts from an NFA

$N = ($
description

Such that

Note :-

The flip side is the same & to containing

→ Other follows

- q_0 is the start state

- often, the form

- Inaccessibility effectively much

$N = (Q_N, \Sigma, \delta_N, q_0, F_N)$. Its goal is the description of a DFA $D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$

Such that $L(D) = L(N)$.

Note :-

The Σ alphabet of the two automata are the same & the start state of 'D' is the set containing only the start state of N .

→ Other components of 'D' are constructed as follows:

- Q_D is the set of subsets of Q_N ; i.e., Q_D is the power set of Q_N . Note that if Q_N has n states, then Q_D will have 2^n states.
- often, not all these states are accessible from the start state of Q_D .
- Inaccessible states can be "thrown away". So effectively, the no of states of D may be much smaller than 2^n .

Fig. (c)

• F_0 is the set of subsets S of Q_N such that $S \cap F_N \neq \emptyset$. That is, F_0 is all sets of N 's states that include at least one accepting state of N .

• For each set $S \subseteq Q_N$ & for each input symbol a in Σ ,

$$\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$$

That is, to compute $\delta_D(S, a)$ we look at all the states p in S , see what states N goes to from p on input a , & take the union of all those states.

Eg 0

Let N be the automata of figure (a) that accepts all strings that end in 0. N 's set of states is $\{q_0, q_1, q_2\}$, the subset construction produces a DFA with $2^3 = 8$ states, corresponding to all the subsets of these 3 states.

$\emptyset$
 $\rightarrow \{q_0\}$
 $\{q_1\}$
 $\{q_2\}$
 $\{q_0, q_1\}$
 $\{q_0, q_2\}$
 $\{q_1, q_2\}$
 $\{q_0, q_1, q_2\}$
 $\rightarrow$ we can

$\{q_0\}$
 $\{q_1\}$
 $\{q_2\}$
 $\{q_0, q_1\}$
 $\{q_0, q_2\}$
 $\{q_1, q_2\}$
 $\{q_0, q_1, q_2\}$

Fig (c) The complete subset construction from figure (a).

	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\emptyset$
$*\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$*\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$*\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$*\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

DEF
NS- $a \rightarrow q_1$
 $a \rightarrow q_1, q_2$

$\rightarrow$ we can invent new names for these states,
eg:- A for $\emptyset$, B for $\{q_0\}$ & so on.

	0	1
A	A	A
$\rightarrow$ B	E	B
C	A	D
* D	A	A
E	E	F
* F	E	B
* G	A	D
* H	E	F

Figure (d)

Renaming the states in above figure.

1. That
2. Set of
3. construction
corresponds

→ of the 8 states in figure (d), starting in the start state B, we can only reach states B, E, & F.

→ The other 5 states are inaccessible from the start state B, & may as well not be there.

→ we often can avoid the exponential-time step of constructing transition-table entries for every subset of states if we perform "Lazy evaluation" on the subsets.

eg :-

→ $\{q_0\}$ is a state of the DFA D . we find that $\delta_D(\{q_0\}, 0) = \{q_0, q_1\}$ &

$$\delta_D(\{q_0\}, 1) = \{q_0\}$$

→ $\{q_0\}$ is new & its transitions may not already been considered. So $\{q_0, q_1\}$ is computed.

$$\begin{aligned} \rightarrow \delta_D(\{q_0, q_1\}, 0) &= \delta_D(q_0, 0) \cup \delta_D(q_1, 0) \\ &= \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\} \end{aligned}$$

δ_D (state)

→ we can

$\delta_D(\{q_0\})$

$\delta_D(\{q_0\})$

→ Thus

we

have

→ The

$$\begin{aligned}
 \delta_D(\{q_0, q_1\}, 1) &= \delta_N(q_0, 1) \cup \delta_N(q_1, 1) \\
 &= \{q_0\} \cup \{q_2\} \\
 &= \{q_0, q_2\}
 \end{aligned}$$

→ we have discovered one new state of D , which is $\{q_0, q_2\}$.

$$\begin{aligned}
 \delta_D(\{q_0, q_2\}, 0) &= \delta_N(q_0, 0) \cup \delta_N(q_2, 0) \\
 &= \{q_0, q_1\} \cup \emptyset \\
 &= \{q_0, q_1\}
 \end{aligned}$$

$$\begin{aligned}
 \delta_D(\{q_0, q_2\}, 1) &= \delta_N(q_0, 1) \cup \delta_N(q_2, 1) \\
 &= \{q_0\} \cup \emptyset \\
 &= \{q_0\}
 \end{aligned}$$

→ Thus, the Subset construction has converged. we know all the accessible states & their transitions.

→ The entire DFA is shown in below figure.

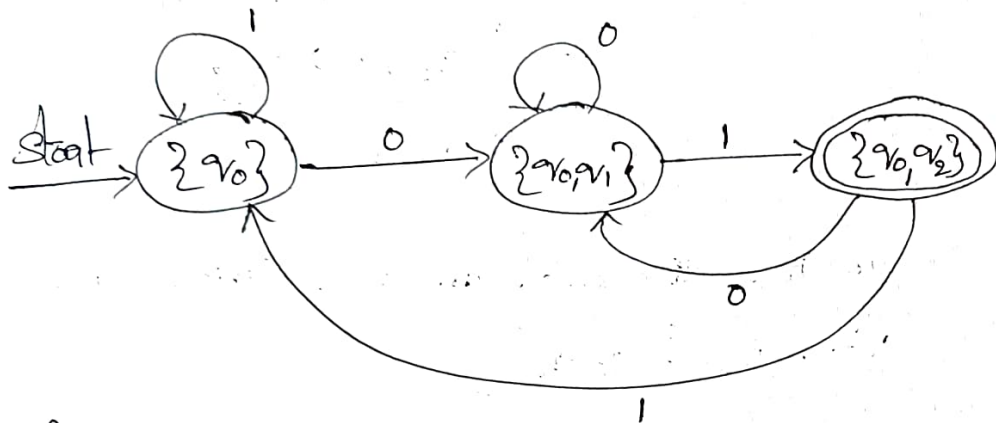


fig (c):- The DFA constructed from the NFA of figure (a).

→ The accepting states of the DFA are those sets that include at least one accepting state of the NFA.

→ we may then conclude that the DFA & NFA accept exactly the same strings, & therefore accept the same language.

eg :- find DFA equivalent to NFA

$$M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\})$$

where δ is given by

→ q_0	q_0, a
q_1	q_1
* (q_2)	

Ans

$$\delta_D(q_0, a)$$

$$\delta_D(q_0, b)$$

$$\delta_D(\{q_0, a\})$$

$$\delta_D(\{q_0, a\})$$

$$\delta_D(q_2, a)$$

$$\delta_D(q_2, b)$$

	a	b
$\rightarrow q_0$	q_0, q_1	q_2
q_1	q_0	q_1
$* (q_2)$	-	q_0, q_1

$$\Sigma = \{a, b\}$$

Ans

NFA

$$\begin{aligned} \delta_D(q_0, a) &= \{q_0, q_1\} \checkmark \\ &= \{q_0, q_1\} \checkmark \end{aligned}$$

$$\delta_D(q_0, b) = \delta_N(q_0, b) = \{q_2\} \checkmark$$

$$\begin{aligned} \delta_D(\{q_0, q_1\}, a) &= \delta_N(q_0, a) \cup \delta_N(q_1, a) \\ &= \{q_0, q_1\} \cup \{q_0\} \\ &= \{q_0, q_1\} \checkmark \end{aligned}$$

$$\begin{aligned} \delta_D(\{q_0, q_1\}, b) &= \delta_N(q_0, b) \cup \delta_N(q_1, b) \\ &= \{q_2\} \cup \{q_1\} \\ &= \{q_1, q_2\} \end{aligned}$$

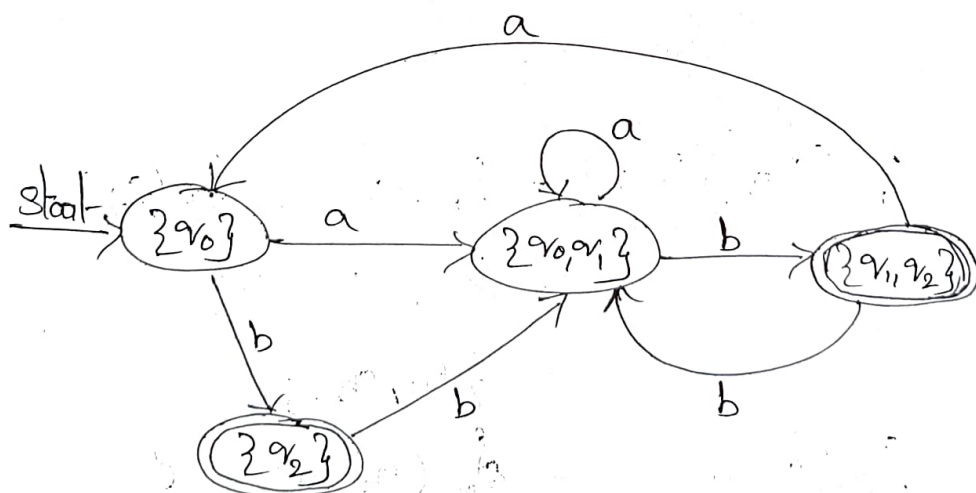
$$\delta_D(q_2, a) = \delta_N(q_2, a) = \emptyset$$

$$\delta_D(q_2, b) = \{q_0, q_1\}$$

$$\begin{aligned}\delta_D(\{q_1, q_2\}, a) &= \delta_N(q_1, a) \cup \delta_N(q_2, a) \\ &= \{q_0\} \cup \emptyset\end{aligned}$$

$$\begin{aligned}\delta_D(\{q_1, q_2\}, b) &= \delta_N(q_1, b) \cup \delta_N(q_2, b) = \\ &= \{q_0, q_1\} \cup \{q_0, q_1\} \\ &= \{q_0, q_1\}\end{aligned}$$

	a	b
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_2\}$
$*\{q_2\}$	$\emptyset$	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_1, q_2\}$
$*\{q_1, q_2\}$	$\{q_0\}$	$\{q_0, q_1\}$



The

①

F

→ ~~U~~ get const

An A

→ it will web

① for

→ there pe document

→ A Sec process

19)

The DFA is defined as

$$D = (Q, \Sigma, \delta, [q_0], F')$$

$$F = \{ [q_2], [q_0, q_2], [q_1, q_2], [q_0, q_1, q_2] \}$$

→ After constructing δ' for $\{q_1, q_2\}$, we don't get any new states & So we terminate the construction of δ .

An Application : Text String :-

→ It is actually an excellent model for several real problems that appear in applications such as web search & extraction of info from text.

(a) String Strings in Text :-

→ the problem is Given a set of words, find all documents that contain one of these words.

→ A Search engine is a popular example of this process. The Search engine uses a particular

technology, called Inverted Indexes, where for each word appearing on the web a list of all the places where that word occurs is stored.

→ Inverted index techniques do not make use of finite Automata, but they also take very large amounts of time for crawlers to copy the web & set up the indexes.

→ The characteristics that make an application suitable for searches that use automata are:

1. The repository on which the search is conducted is rapidly changing. For example:

(a) Every day, news analysts want to search the day's on-line news articles for relevant topics.

(b) A "Shopping robot" wants to search for

the current
its client

2. The document
catalog

(b) NFA

→ Suppose

; which we
want to
work.

→ There
recognize

1. There is
itself an

eg. $\frac{1}{2}$

Gray p
examined